ATTESTOR — MASTER BUILD PROMPT

> **How to use this:** Paste this entire document as the opening instruction to your coding agent (Google Antigravity, in Manager view). It is written for an AI agent, not a human. Do not summarise it — paste it whole. Then work through the milestones in order.

---

1. YOUR ROLE

You are a senior full-stack engineer building a production-grade prototype in a 2-day hackathon (Startup Buildathon India 2026, Google DeepMind + Google Cloud).

You will be judged on: product potential, market viability, **technical architecture**, and **depth of Google AI + Google Cloud usage**. The demo is 7 minutes plus 3 minutes of hostile technical Q&A, in front of both engineers and non-technical business jurors.

Optimise for: **a working end-to-end demo with real evidence**, not feature count. A narrow product that works completely beats a broad product that works partially. If you run out of time, cut rules, never cut evidence quality.

---

2. WHAT YOU ARE BUILDING

Product name: Attestor **Tagline:** Proof your product behaves.

The problem. 83% of organisations now require a documented Accessibility Conformance Report (ACR/VPAT) before they will purchase software (Level Access / Sapio Research, September 2026, n=2,530). Today that report is written by a human consultant over 2–4 weeks for USD 1,900–3,700, and it is out of date at the next deployment. Automated scanners like axe-core cannot replace it — they reliably test only ~13% of WCAG success criteria, because the rest require judgement: *is this alt text meaningful? does focus order match visual order? is this error actually announced? can a keyboard user escape this modal?*

The product. Attestor takes a live URL, sends AI agents to operate the application the way a keyboard-only and screen-reader user must, captures trace-level evidence at every step, judges that evidence against WCAG 2.2 AA and EN 301 549 using Gemini, and emits a **signed Accessibility Conformance Report in which every claim hyperlinks to the screenshot, accessibility-tree node and reproduction steps that prove it** — stamped with the Git commit SHA, regenerable on every build.

Positioning (use this in UI copy): *Vanta proves your company has controls. Attestor proves your product behaves.*

What this is NOT. Not a chatbot. Not a dashboard. Not an accessibility overlay widget (those are legally discredited — the FTC fined accessiBe USD 1M in 2025). Not a wrapper around axe-core. The deliverable is **the attested report and the evidence chain behind it**, not a list of defects.

---

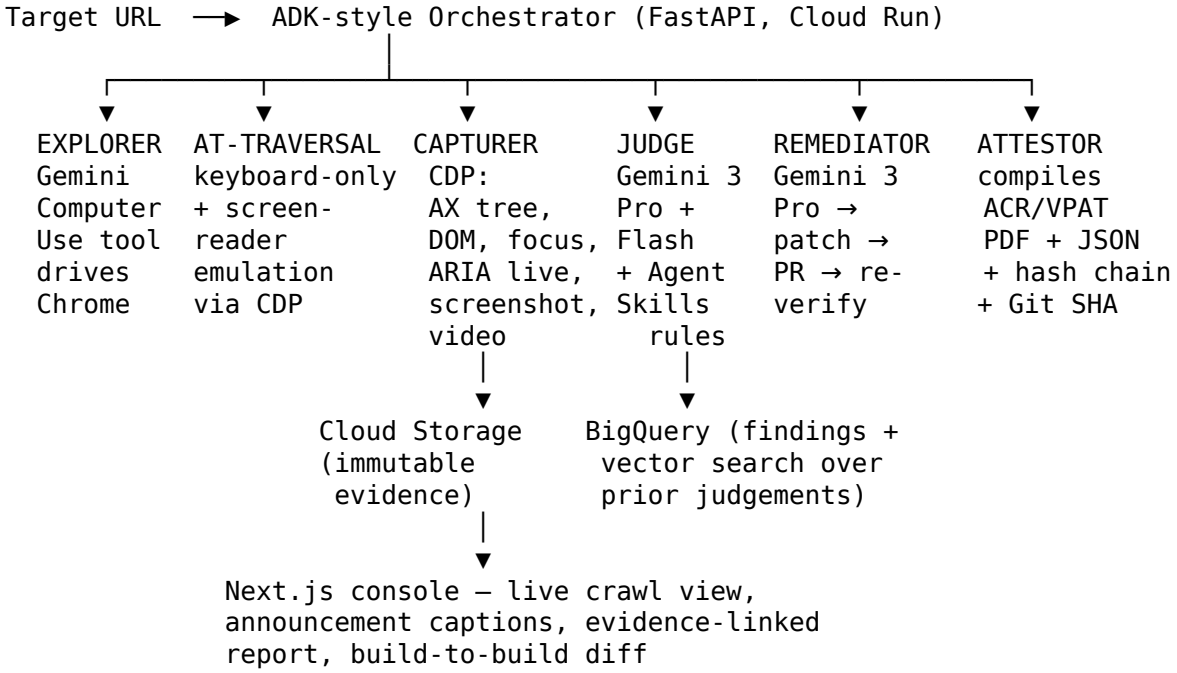
3. NON-NEGOTIABLE CONSTRAINTS

1. **Google-only for AI.** Every model call goes to Google: Gemini 3 Pro, Gemini 3 Flash, and the Gemini Computer Use tool. No OpenAI, no Anthropic, no Hugging Face inference, no third-party AI SaaS. This is a hard competition requirement. *Clarification:* Chrome, the Chrome DevTools Protocol, Playwright (as a CDP driver only), Next.js, FastAPI and Postgres are infrastructure, not AI vendors. They are permitted. The constraint applies to **models and AI tooling**.
2. **All three named Google tools must be genuinely used, not name-dropped:** Google AI Studio (prompt development), Google Antigravity (build environment), Google Agent Skills (rule packaging — this one must be *architecturally* load-bearing, not cosmetic).
3. **Deploy to Google Cloud Run.** Judges must be able to open a URL.
4. **No scraping of sites that forbid it.** Demo only against sites we own, public demo apps, or a URL a juror volunteers.

5. **No real user data.** Evidence capture may record screenshots — never run against an authenticated production account containing third-party personal data during the demo.

---

4. ARCHITECTURE



Repository layout — create exactly this:

```
attestor/
├── README.md
├── docker-compose.yml
├── .env.example
├── skills/                          # Google Agent Skills — the rule packs
│   ├── focus-order/SKILL.md
│   ├── meaningful-alt-text/SKILL.md
│   ├── no-keyboard-trap/SKILL.md
│   ├── error-identification/SKILL.md
│   ├── name-role-value/SKILL.md
│   ├── focus-visible/SKILL.md
│   ├── link-purpose/SKILL.md
│   ├── form-labels/SKILL.md
│   ├── heading-structure/SKILL.md
│   ├── skip-link/SKILL.md
│   ├── status-messages/SKILL.md
│   ├── page-title/SKILL.md
│   ├── language-of-page/SKILL.md
│   ├── reflow/SKILL.md
│   └── consistent-navigation/SKILL.md
├── corpus/                          # normative text for grounding
│   ├── wcag22.md
│   ├── en301549.md
│   └── aria-apg.md
├── backend/
│   ├── main.py                      # FastAPI + SSE stream to frontend
│   ├── orchestrator.py              # run plan, agent dispatch
│   ├── agents/
```

```
│   │   ├── explorer.py             # Gemini Computer Use loop
│   │   ├── traversal.py            # keyboard-only + SR emulation
│   │   ├── capturer.py             # CDP evidence capture  ← BUILD FIRST
│   │   ├── judge.py                # Gemini 3 Pro/Flash + skills
│   │   ├── remediator.py           # patch + PR
│   │   └── attestor.py             # ACR compilation + signing
│   ├── skills_loader.py            # parse SKILL.md, progressive disclosure
│   ├── evidence.py                 # content-addressed store + hash chain
│   ├── models.py                   # Pydantic schemas
│   └── report/
│       ├── acr_template.html
│       └── render.py               # HTML → PDF
└── frontend/                       # Next.js + TypeScript + Tailwind
    ├── app/page.tsx                # URL input + launch
    ├── app/run/[id]/page.tsx       # live crawl + captions
    ├── app/report/[id]/page.tsx    # evidence-linked ACR
    └── components/
        ├── LiveBrowser.tsx         # screenshot stream
        ├── AnnouncementCaptions.tsx # ← the emotional core of the demo
        ├── CriterionCard.tsx
        └── EvidenceDrawer.tsx
```

---

5. COMPONENT SPECIFICATIONS

5.1 Capturer — `backend/agents/capturer.py` — BUILD THIS FIRST

This is the product. Everything else is replaceable; this is not.

Connect to Chrome over the DevTools Protocol. At **every interaction step**, capture one `EvidenceStep` record:

Field	How to get it
`ax_tree`	`Accessibility.getFullAXTree` — filter to nodes in the viewport before storing
`focused_node`	`Runtime.evaluate` on `document.activeElement`, then `Accessibility.getPartialAXTree` for its backendNodeId
`accessible_name`	from the AX node's `name` property — this is what a screen reader speaks
`role`	AX node `role`
`dom_snapshot`	`DOMSnapshot.captureSnapshot`
`computed_styles`	include outline/box-shadow on the focused element (needed for focus-visible)
`announcements`	inject a `MutationObserver` on `[aria-live]`, `[role=alert]`, `[role=status]` via `Page.addScriptToEvaluateOnNewDocument`; buffer and drain each step
`screenshot`	`Page.captureScreenshot` (JPEG q70 — keep the bundle small)
`network`	`Network.responseReceived` summaries
`action`	what we did to get here: `{type: "key", key: "Tab"}` etc.
`timestamp`, `url`, `step_index`	

Screen-reader emulation. Do not shell out to NVDA. Synthesize the announcement from the AX node, in this order, exactly as a screen reader would:

```
f"{accessible_name}, {role}{', ' + state if state else ''}"
# e.g. "Pay now, button"  |  "button"  ← the second one is a failure
# e.g. "Email, edit, required, invalid entry"
```

Store it as `synthesized_announcement`. This string is what you render as captions in the UI, and it is the single most persuasive thing in the demo.

Keyboard traversal loop:

```
1. Press Tab (Input.dispatchKeyEvent)
2. Read document.activeElement
3. If focus did not move → candidate keyboard trap
4. If focus moved to an element outside the viewport → candidate focus-order issue
5. If focus returned to an element already visited → cycle; record and break
6. Capture EvidenceStep
7. Repeat up to 150 steps or until focus cycles
```

Evidence integrity. Each step is serialised deterministically, SHA-256 hashed, and chained: `hash(step_n) = sha256(canonical_json(step_n) + hash(step_n-1))`. Store blobs content-addressed in Cloud Storage. The report references content hashes, so a report can never drift from the evidence it cites. Sign the final manifest.

5.2 Explorer — `backend/agents/explorer.py`

Use the **Gemini Computer Use tool** (GA in the Gemini API since 30 October 2026) with the `google-genai` Python SDK. Loop:

```
1. Screenshot current page → send to model with the goal
   ("Sign up for an account", "Add an item to the cart and reach checkout")
2. Model returns a UI action (click at x,y / type text / press key / scroll)
3. Execute it in Chrome via CDP
4. Send the resulting screenshot back
5. Repeat until goal reached or 25 steps
```

Record the discovered flow as an ordered list of actions so the AT-traversal agent can replay the same journey keyboard-only. Handle safety confirmations the model requests. Always cap steps and wall-clock time.

5.3 Judge — `backend/agents/judge.py`

For each `(EvidenceStep, criterion)` pair: 1. Load the criterion's **Agent Skill** (`skills/<name>/SKILL.md`) via `skills_loader.py`. 2. Retrieve the normative text for that criterion from `corpus/`. 3. Call Gemini with the skill body, the normative text, the AX node, the synthesized announcement, the computed styles and the screenshot. 4. Demand structured JSON output.

Model routing — be deliberate, and be ready to defend it: - `gemini-3-pro-preview` with `thinking_level: "high"` → judgement-heavy criteria (meaningful alt text, focus order, error identification, link purpose). - `gemini-3-flash` → bounded, high-volume per-node checks (page title, language, form labels, heading structure). - Log tokens and latency per call. **Produce a cost-per-report table.** A juror will ask; having the number is worth more than the number itself.

Required output schema:

```
{
  "criterion": "2.4.3 Focus Order",
  "verdict": "supports | partially_supports | does_not_support | not_applicable",
  "confidence": 0.0,
  "reasoning": "one paragraph a non-expert can follow",
  "normative_citation": "verbatim quote from WCAG/EN 301 549",
  "evidence_step_ids": ["..."],
  "user_impact": "plain English: what a real person cannot do",
  "suggested_fix": "concrete code change",
```

```
  "requires_human_review": true
}
```

Hard rules for the Judge: - Never claim `supports` with confidence < 0.85 — downgrade to `requires_human_review: true`. A false "Supports" in a procurement document is a misrepresentation with legal weight, and saying so out loud in the demo wins credibility. - `normative_citation` must be retrieved text, never generated. If no citation is retrieved, the finding is discarded. - `user_impact` must name a person and an action: *"A screen-reader user cannot tell which of the four 'Read more' links goes to the refund policy."*

5.4 Agent Skills — `skills/*/SKILL.md`

This is a judged differentiator. Make it real. Format:

```
---
name: meaningful-alt-text
description: Use when evaluating whether an image's accessible name conveys
  the information the image carries. Covers WCAG 2.2 criterion 1.1.1 Non-text Content.
criterion: "1.1.1"
level: "A"
model: gemini-3-pro-preview
---

# Meaningful alternative text

## What you are judging
Not whether alt text EXISTS — a scanner already checked that. Judge whether the
accessible name conveys the same information a sighted user gets from the image.

## Procedure
1. Look at the screenshot region for this image.
2. Read the accessible name from the AX node.
3. Ask: if the image vanished and only this text remained, would the user lose information?
4. Decorative images correctly have an empty name — that is a PASS, not a failure.

## Fails
- Filename as alt text: "IMG_4471.png", "hero-banner-final-v2.jpg"
- Generic: "image", "photo", "icon", "graphic"
- Redundant: alt repeats adjacent visible text verbatim
- A chart whose alt is "chart" rather than its trend or value
- A product photo whose alt omits the colour/variant the page is selling

## Passes
- Decorative flourish with alt=""
- "Line chart: monthly revenue rising from ₹2L in January to ₹9L in June"
- Icon button whose accessible name is its action, e.g. "Delete item"

## Edge cases
- Image inside a link: the name must describe the DESTINATION, not the picture.
- CAPTCHA images: must describe purpose, not content.
- Text rendered as an image: alt must contain that exact text.
```

Write all 15 to this standard. `skills_loader.py` must implement **progressive disclosure** — load only each skill's frontmatter `description` when matching, then load the full body only for the skill that applies. Say this explicitly in the demo: it keeps context small and lets a non-engineer extend the product by writing Markdown.

5.5 Attestor — `backend/agents/attestor.py`

Compile findings into an ACR on the **VPAT 2.5 / EN 301 549** structure. Required properties: - One row per success criterion: Supports / Partially Supports / Does Not Support / Not Applicable + remarks. - **Every remark hyperlinks to its evidence** (screenshot + AX node + reproduction steps + step index). - Header: target URL, run timestamp, Git commit SHA, Attestor version, evidence manifest hash, signature. - Footer: honest methodology statement — which criteria were machine-judged, which need human counter-sign, and the confidence threshold used. - Export both PDF and JSON. **PDF must be text-based, never a rasterised image.**

5.6 Frontend

Three screens. The middle one wins the hackathon.

1. **Landing** — single URL input. One button: *Attest this product*. Nothing else.
2. **Live run** — split view:
   - left: streaming screenshots with the current focus ring highlighted
   - right: **`AnnouncementCaptions.tsx`** — the synthesized screen-reader announcements scrolling as captions, large type, one line per Tab stop. When the announcement is just `"button"` with no name, render it **in red**. When focus fails to move, render `⚠ KEYBOARD TRAP` and hold it on screen.
   - bottom: live criterion verdicts appearing as they resolve.
3. **Report** — the ACR. Every verdict row expands into an evidence drawer: screenshot with the element outlined, the AX node JSON, reproduction steps, the normative citation, and the suggested fix. Plus a **diff toggle** comparing against the previous run.

Accessibility of our own product is non-negotiable and will be checked by a juror: semantic HTML, visible focus indicators, keyboard operable, correct heading hierarchy, live regions for status updates. **Run Attestor against Attestor and show the result.** That is the best possible closing line.

---

6. BUILD ORDER

Do not deviate. Each milestone must run end-to-end before the next begins.

M	Milestone	Done when
M0	Repo, `.env`, Gemini API key, Chrome headless launching, FastAPI serving, Next.js rendering	`GET /health` returns OK and Chrome opens a page
M1	**Capturer.** Tab through a real page, capture full EvidenceStep per stop, synthesize announcements, hash-chain, write to disk	JSON file with 30+ steps, each with AX node + screenshot + announcement
M2	**Live UI.** Stream M1 to the browser over SSE with captions rendering	You can watch it tab through a site live and read the announcements
M3	**Judge + 5 Skills.** focus-order, meaningful-alt-text, no-keyboard-trap, name-role-value, error-identification	5 verdicts with citations and evidence IDs on a real site
M4	**ACR export.** HTML → text-based PDF, every claim linked to evidence	Downloadable PDF that opens and is Ctrl-F searchable
M5	**Explorer.** Computer Use drives a signup/checkout flow, then traversal replays it keyboard-only	Agent completes a goal unaided; traversal replays the same journey
M6	**Remediator.** One real fix → PR → re-run shows the criterion flip	Green diff on screen

M	Milestone	Done when
M7	**Deploy + 10 more skills + cost table + run Attestor on Attestor**	Public Cloud Run URL a juror can open

If you are behind at the end of Day 1, cut M5 and M6 before cutting anything else. A flawless M1–M4 is a winning demo. M5 is impressive but optional; M6 is a nice-to-have.

---

7. THE DEMO — build toward this exact sequence

Seven minutes. Rehearse it three times on Day 2 evening.

1. **(0:00)** *"Give me any URL."* Take one from a juror. If nobody volunteers, use a pre-agreed real site.
2. **(0:30)** Launch. Agent begins exploring. Captions start scrolling.
3. **(1:30) The moment.** Captions hit a bare `"button"` in red. Say: *"A blind user is about to pay for something. Their screen reader just said 'button.' That's all they get."* Then a keyboard trap fires and the caption freezes on ⚠ KEYB0ARD TRAP. Say nothing for two seconds.
4. **(3:00)** The report renders. Click one "Does Not Support" row → evidence drawer opens with screenshot, AX node, repro steps, normative citation.
5. **(4:00)** Hit the diff toggle: *"Here's the same report against last night's build. This criterion regressed in commit a3f91c."*
6. **(4:45)** Show the remediation PR, and the re-run flipping the criterion to Supports.
7. **(5:30)** Architecture slide. Name every Google component out loud. Show the Pro-vs-Flash cost-per-report table.
8. **(6:30)** Closing: *"We ran Attestor against Attestor. Here's our own report."*

Fallback: record this entire run as a video on Day 2 afternoon. Venue WiFi fails. Have it and never need it.

---

8. PITFALLS — read before writing code

1. **Do not build a scanner.** If you find yourself reimplementing axe-core, stop. The value is judgement over the criteria axe *cannot* test, and the evidence chain. Optionally run axe for the easy 13% and spend Gemini only on the hard 87% — then say so, because it demonstrates engineering judgement.
2. **Evidence capture is the hard part, not the AI.** Budget accordingly. A beautiful judge over sloppy capture is a demo that falls apart under questioning.
3. **Screenshots will blow up your payloads.** JPEG q70, downscale to 1280px wide, store blobs separately and pass references.
4. **Gemini 3 is recent.** Verify model IDs and pricing at `ai.google.dev/gemini-api/docs/models` and `/pricing` on build day. Do not hard-code a price onto a slide without re-checking.
5. **Multi-turn tool calling with Gemini 3 requires passing thought signatures back.** Read the docs before debugging a mystery.
6. **Never let a live WebSocket be the only path to the money shot.** Cache a successful run and be able to replay it instantly.
7. **Do not say "Gemini Omni."** It is not a documented Google product name, despite appearing on the event site. Use real model IDs. Precision in front of Google jurors is worth more than echoing marketing copy.
8. **Our own product must be accessible.** A juror will Tab through it. If Attestor is inaccessible, the pitch dies on the spot.
9. **Never claim full WCAG conformance from a machine run.** Always display the confidence threshold and which criteria require human counter-sign. Honesty here is a feature, not a weakness — and it's also the paid tier.

---

9. DEFINITION OF DONE

- ☐ Public Cloud Run URL a juror can open on their phone
- ☐ Accepts an arbitrary URL and produces a report without code changes
- ☐ ≥15 criteria implemented as Agent Skills with progressive disclosure
- ☐ Every verdict traces to immutable, hash-chained evidence
- ☐ Downloadable, text-based, searchable ACR PDF
- ☐ Build-to-build diff works on two real runs
- ☐ At least one remediation PR demonstrated
- ☐ Cost-per-report table for Gemini 3 Pro vs Flash
- ☐ Attestor's own ACR, generated by Attestor, published
- ☐ Attestor itself passes keyboard-only operation
- ☐ Fallback demo video recorded
- ☐ README with architecture diagram and a one-command local run

---

10. FIRST INSTRUCTION

Start with **M0 and M1 only**. Build the CDP evidence capture: open Chrome, Tab through `https://example.com`, and for each focus stop emit an `EvidenceStep` containing the accessibility-tree node, the synthesized screen-reader announcement, the focus position, the computed outline style, and a screenshot — hash-chained and written to `./runs/<run_id>/`.

Show me that JSON output before writing anything else.